

AAP — Document technique complet

A. Vision du projet

AAP signifie AI Administration Platform. Le projet vise à simplifier la conception et l'utilisation des interfaces d'administration d'applications en exploitant des données réelles et, dans les phases avancées, des capacités IA pour interroger, surveiller et diagnostiquer les systèmes.

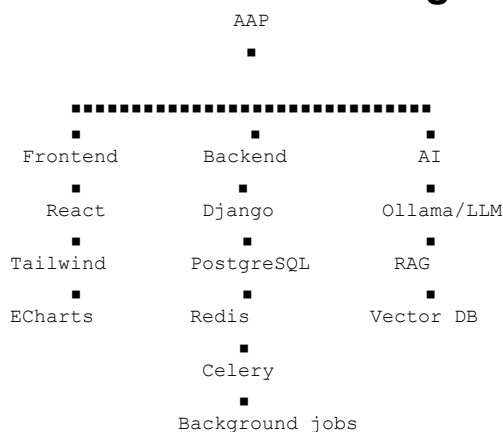
B. Objectif de développement

Le projet sert aussi de parcours technique pour approfondir Python/Django, PostgreSQL, Redis, systèmes distribués, sécurité, LLM, RAG, vector databases, tool calling et MCP, afin de construire un profil Python + AI backend.

C. Principes du produit

AAP doit rester dynamique et ne pas être limité à un secteur particulier. Dans les phases avancées, la plateforme s'appuiera sur les tables et données du client plutôt que de copier ses données dans AAP. Les opérations IA prévues sur les données clientes seront orientées lecture seule.

D. Architecture cible à long terme



E. Roadmap

AUG–SEP : design, architecture et fondations. OCT–DEC : cœur produit. JAN–MAR : IA et backend avancé.
APR–MAY : production, documentation et PFE.

1. Backend — fondations avant le CRUD

Sécurité

Avant toute fonctionnalité métier : JWT access/refresh tokens ; OAuth 2.0 / Google ; vérification email ; hashage des mots de passe avec Argon2 ou bcrypt ; CORS strict ; CSRF selon le mode d'authentification ; rate limiting ; protection brute-force ; validation des entrées ; RBAC ; séparation utilisateur/admin ; sécurité des headers ; gestion centralisée des exceptions ; logs de sécurité ; audit logs ; secrets via variables d'environnement ; aucune donnée sensible dans les logs ; pagination des endpoints ; limites de payloads ; contrôle des uploads ; HTTPS en production ; cookies HttpOnly, Secure et SameSite ; rotation/révocation des refresh tokens.

Redis

Redis sert au rate limiting, cache, sessions/token blacklist si nécessaire, données temporaires, OTP/email verification codes et tâches courtes. Celery pourra l'utiliser plus tard.

Frontend

```
↓
Traefik

Django / DRF
↓
Redis
↓
PostgreSQL
```

Redis ne remplace pas PostgreSQL. PostgreSQL reste la source de vérité.

2. Principe fondamental : le frontend ne possède pas les données métier

Le frontend ne doit pas contenir en dur les données administrables : nom de l'application, description, images, plans, rôles, permissions, contenu des pages, messages métier ou configuration. Le frontend devient principalement un client de l'API. Les éléments purement techniques peuvent rester dans le code, mais les textes devraient idéalement passer par i18n.

3. Hashing et chiffrement

Les mots de passe sont hashés avec Argon2 ou bcrypt et ne doivent pas être récupérables. Les données récupérables mais sensibles, comme customer database credentials, API keys et OAuth secrets, devront être chiffrées et non hashées.

4. Base de données

PostgreSQL est la base principale avec Django ORM. Les données persistantes sont dans PostgreSQL ; Redis est une couche complémentaire de cache et de données temporaires.

5. Architecture Django

```
backend/
  config/
  apps/
  accounts/
  authentication/
  users/
  administration/
  notifications/
  audit/
  core/
  manage.py
  requirements/
  .env
```

Les modules AI/customer-database ne sont pas développés dans cette première étape ; l'architecture doit pouvoir les accueillir plus tard.

6. Authentification

Inscription : validation hash du mot de passe → création utilisateur → email de vérification → activation. Login : email + mot de passe → vérification access token + refresh token. Google : React → Google OAuth → Django → vérification du token Google → utilisateur JWT.

7. Email verification

```

POST /api/auth/register/
↓
create user
↓
is_verified = false
↓
generate verification token
↓
send email

GET /api/auth/verify-email/<token>/
↓
is_verified = true

```

8. Rôles

Première version : SUPER_ADMIN, ADMIN, USER. Ne pas créer trop de rôles au début. COMPANY_ADMIN, EMPLOYEE ou VIEWER pourront être ajoutés si nécessaire.

9. CRUD initial

User : Create, Read, Update, Delete avec permissions. Admin : lister les utilisateurs, consulter un utilisateur, modifier son statut ou rôle, activer/désactiver un compte et consulter l'activité de base. User : créer son compte, vérifier son email, se connecter, se déconnecter, consulter/modifier son profil et changer son mot de passe.

10. Seed admin

Utiliser un management command Django, par exemple `python manage.py seed_data`. Le compte administrateur est créé depuis des variables d'environnement et non avec un mot de passe réel dans le repository. Le mot de passe est hashé par Django.

11. Frontend — composants réutilisables

```

frontend/src/
  reusedComponents/
  ■   Alert/
  ■   Notification/
  ■   ConfirmDialog/
  ■   Button/
  ■   Input/
  ■   Form/
  ■   Icon/
  ■   Breadcrumb/
  ■   Sidebar/
  ■   Navbar/
  ■   Modal/
  ■   Loader/
  ■   EmptyState/
  ■   ErrorState/
  ■   Pagination/
  ■   DataTable/
  ■   Charts/
  ■   LanguageSwitcher/
  ■   ThemeSwitcher/

  pages/
  layouts/
  features/
  services/
  store/
  hooks/
  routes/
  i18n/
  types/
  utils/
  assets/

```

12. Redux Toolkit

Redux Toolkit gère surtout l'état UI/global : thème, langue, authentification/permissions et sidebar. Les données serveur pourront être gérées avec TanStack Query plutôt que de mettre toutes les données API dans Redux.

13. i18n

i18n/

```
... fr/
  ... translation.json
... en/
  ... translation.json
```

14. Charts

Pour AAP, utiliser Apache ECharts plutôt que Recharts. Les graphiques seront encapsulés dans reusedComponents/Charts/

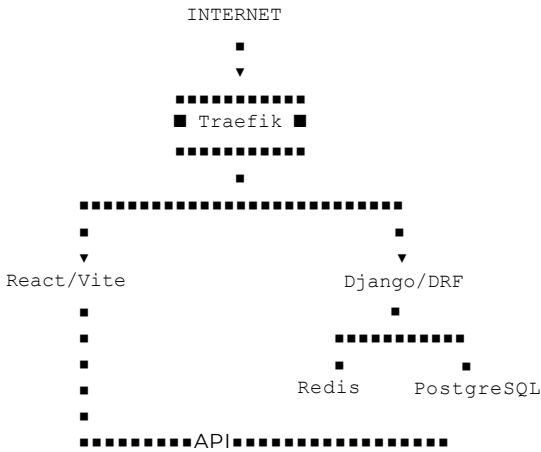
reusedComponents/Charts/

```
... LineChart/
... BarChart/
... AreaChart/
... PieChart/
... ChartCard/
```

15. Pages initiales

Public : Home, Login, Register, Verify Email, Forgot Password. User : Dashboard, Profile, Settings. Admin : Dashboard, Users, User details, Notifications, Audit.

16. Architecture complète de la première version



17. Ce qu'on ne fait pas maintenant

Phase1 : React, Vite, TypeScript, Tailwind, Redux Toolkit, i18n, ECharts, Django, DRF, JWT, Google OAuth, PostgreSQL et Redis. Phase 2 : Docker, Traefik, Celery, CI/CD et GitHub Actions. Phase 3 : LLM, Ollama, RAG, embeddings, vector DB, tool calling et MCP. Phase 4 : connexions aux bases clientes, monitoring, logs, génération SQL en lecture seule, diagnostics IA et observabilité.

18. Première cible

AAP

```
■
...
■ Register
  ↓
```

```

■■■ Email verification
■
  ↓
■■■ Login
■
  ↓
■■■ JWT
■
  ↓
■■■ User dashboard
■
■■■ Admin dashboard
    ■■■ Users
    ■■■ User details
    ■■■ Account management
    ■■■ Notifications
    ■■■ Audit logs

```

Objectif : une base solide avec Security + PostgreSQL + Redis + Django REST + JWT/OAuth + React/TypeScript + Redux Toolkit + i18n + composants réutilisables, sans paiement, marketplace ou IA dans cette première étape.